

Message Notification Router — Agent Workflow & Engineering Report

HackerRank Orchestrate, August 2026 — AI Judge interview preparation. Written from the session's actual build/test/decision history.

1. Executive Summary

The challenge: build a personalized notification router for a WhatsApp-style platform. For every incoming message (text, image poster/screenshot, or voice note), decide **notify** (interrupt now), **digest** (show later), or **mute** (suppress), plus a **message_type**, a grounded **reason**, a calibrated **confidence**, and supporting **evidence_message_ids** from the user's own history — personalized per receiving user, not per message content alone.

Approach: a local preprocessing layer (SQLite-backed context assembly, cached image/voice analysis) feeds a Claude Sonnet 5 classification step constrained to a JSON schema. The prompt and schema were refined over 8 documented iterations, each driven by reading actual miss cases rather than guessing, and validated against three independent signals: the official 30 solved examples, a genuinely held-out 19-case synthetic test set (text + generated images + generated voice notes) built specifically to catch over-fitting to the visible examples, and targeted cheap re-checks on known-hard cases.

action accuracy	93.33% (30-sample eval)
message_type accuracy	93.33% (30-sample eval)
evidence F1	~48–52% (across evals)
output.csv	110/110 rows, schema-valid, no fallback rows
model	claude-sonnet-5 (vision + classification), local faster-whisper (ASR)

The most interview-relevant fact may be a negative result: a plausible-sounding prompt idea (a `key_phrase` grounding field) was implemented, tested, found to *regress* accuracy (93.33% → 86.67%), and reverted — kept in this report because catching and reverting a bad idea is as much a signal of process quality as any accepted improvement.

2. Problem Recap

- Input:** `dataset/messages.csv` — one row per incoming message (sender, conversation type, timestamp, text or media reference).
- Output:** `message_id`, `action`, `message_type`, `reason`, `confidence`, `evidence_message_ids` — exact columns, exact order, one row per input message.
- Personalization is the core requirement:** the same message content can be notify for one user and mute for another, based on that user's relationships, history, and preferences.
- Multimodal:** text, image posters/screenshots (needs OCR/vision), and voice notes (needs ASR) must all be reasoned over, not defaulted to a generic type.
- Constraints:** no hardcoded labels or file-specific answers; deterministic where possible; secrets only via environment variables; must include an evaluation workflow.

- **Context data provided:** per-user notification behavior, group metadata and membership, business account trust signals and per-user relationship history, past messages and how the user reacted to them (opened/replied/dismissed/muted/reported), daily notification load.

3. System Architecture

The pipeline separates decision logic from execution logic across five stages, each a separate, independently testable module rather than one large prompt doing everything:

Stage	Module	What it does
1. Ingestion	db.py	Loads every dataset/*.csv into a local SQLite DB, rebuilt fresh each run. dataset/ stays strictly read-only except for output.csv.
2. Media normalization	image_pipeline.py audio_pipeline.py	Images: MIME-sniffed (not trusted by extension), decoded with Pillow, resized, re-encoded as JPEG, then sent to Claude for description + OCR + doc-type. Voice: transcribed locally with faster-whisper behind a swappable provider interface (the Messages API has no direct audio input type).
3. Context assembly	router.py pull_context()	Per message: receiving user's profile, the specific group-membership or business-relationship row, cached media analysis, today's notification load, and a relevance-ranked slice of that user's own message history with reactions — never the full CSVs.
4. Classification	router.py classify_message()	Claude Sonnet 5 call constrained to a JSON schema. Internal-only fields (sender_trust, urgency_signal, risk_signal, repetition_signal, type_candidates) are filled BEFORE the final decision fields, forcing explicit intermediate reasoning.
5. Output + evaluation	main.py, to_output_row() code/evaluation/	Every message gets exactly one output.csv row, even on failure (safe fallback, never a missing row). A separate evaluation workflow scores predictions against the 30 solved examples.

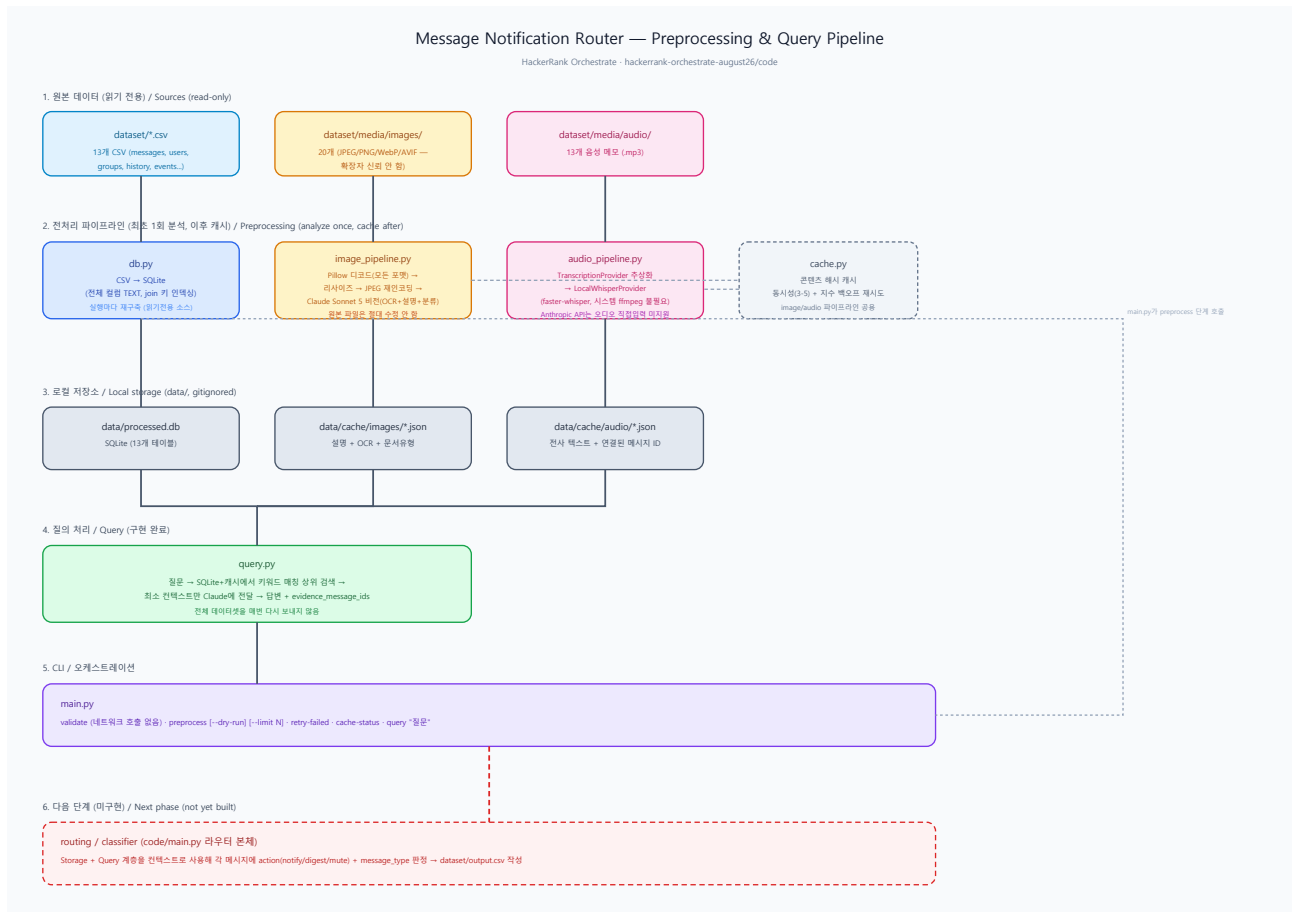


Figure 1. Preprocessing, storage, query, and routing layers (docs/pipeline.svg — the routing/evaluation layers described above sit downstream of what this diagram shows).

4. Multimodal Handling — Notable Details

- **Format mistrust paid off:** MIME-sniffing all 20 images (not just the 3 the problem flagged) found several “.jpg” files that were actually PNG, in addition to the WebP/AVIF cases. Always decoding-then-re-encoding to JPEG handled every case uniformly with no per-format branching.
- **No audio input type exists** in the Anthropic Messages API as of this build — confirmed via the API reference before choosing an approach, not assumed. Local faster-whisper (PyAV-bundled, no system ffmpeg needed) was used behind a TranscriptionProvider abstraction so a hosted ASR API could be swapped in later with one class change.
- **Content-hash caching** (SHA-256 of file bytes) means a byte-identical file is never re-analyzed, even across separate runs — renaming doesn't force reprocessing, but any real change does, automatically.
- **Bounded concurrency + exponential-backoff retry** is shared infrastructure (cache.py) used by both the image and audio pipelines, not duplicated per pipeline.

5. Classification Design — Forcing Intermediate Reasoning

Reviewing early results surfaced two patterns: **digest**→**notify** was the single most common miss direction, and wrong predictions carried visibly lower confidence than correct ones (0.70 vs 0.85 average, across 49 combined evaluation cases). The fix: extend the JSON schema with internal-only fields the model must fill *before* the final decision — never written to output.csv, but forcing the decision to follow from explicit signals rather than being decided first and rationalized afterward.

Internal field	Purpose
sender_trust	verified_trusted_relationship → unverified_or_new_no_relationship (5 levels)
urgency_signal	Does the text itself state or imply urgency, either way? Catches phrases like “no rush” that a plan/time mention alone would override.
risk_signal	none → clear_scam_pattern — deception/credential/payment-extraction signals
repetition_signal	How this message relates to the user's history with this sender/topic
type_candidates	1–3 candidate message_types with relative likelihood, considered before committing to the single best fit

Plus one explicit rule targeting the observed bias: *“When notify and digest are genuinely close, default to digest — uncertainty alone is not justification to interrupt.”* Combined effect, validated on the 30-sample eval: **action accuracy 86.67% → 93.33%, message_type 90% → 93.33%, no regression** on the 26 previously-correct cases.

6. Key Design Decisions

Selected entries from the running decision log (full log in [CLAUDE.en.md/CLAUDE.ko.md §8](#)) — each includes the alternative that was rejected and why:

Decision	Reason / rejected alternative
Always re-encode images to JPEG after resize, regardless of source format.	Several “.jpg” files in the dataset were actually WebP/AVIF/PNG. Decode-then-re-encode handles every format uniformly (including AVIF, which the vision API doesn't accept as a media_type) with no per-format branching. Rejected: trusting the file extension.
Local faster-whisper instead of a hosted ASR API.	Confirmed the Messages API has no audio content-block type, so transcription is mandatory regardless. Local avoids a second API key/cost and PyAV avoids the “no ffmpeg” blocker on this machine. Rejected: OpenAI Whisper API (unnecessary cost/key for no accuracy requirement here).
Python 3.12 venv instead of the system's default Python 3.14.	ctranslate2 (faster-whisper's backend) is a compiled wheel that lags new Python releases by months; 3.14 was too recent to trust in a time-boxed hackathon. Verified 3.12 installed cleanly before committing. Rejected: 3.14 (real risk of no prebuilt wheel).
Routing decisions cached by message_id, not content-hash (unlike media).	messages.csv is static for the run's duration and message_id already uniquely identifies each row; hashing the full joined context (8+ tables) would be complexity with no benefit. Rejected: content-hash keying (over-engineered here); no caching (re-pays on every test run).
evidence_message_ids retrieval is a hand-scored heuristic (sender > group > business > recency, capped at 12), not embeddings.	Dataset is small enough (at most a few hundred history rows per user) that a fast, explainable heuristic is sufficient and keeps token cost low. Rejected: full-history dump (token-expensive, dilutes signal); embedding search (unneeded complexity at this scale).
On classification failure after retries, emit a safe digest/unknown fallback row rather than skipping the message or crashing the run.	The submission contract requires exactly one row per message; a missing row is worse than a clearly-labeled low-confidence placeholder, and the cache still marks it failed so retry-failed fixes it for real later. Rejected: omitting the row; crashing the whole run.

Decision	Reason / rejected alternative
message_type disambiguation rules were derived by reading the actual text of every miss in the worst-performing categories, not by guessing plausible-sounding definitions.	Category boundaries in this dataset are a labeling convention, not something inferable from category names alone (e.g. a business-sender appointment reminder being labeled “event” isn’t obvious without seeing that example). Rejected: generic textbook category definitions without checking real examples.
A second, genuinely held-out synthetic test set was built rather than trusting the 30-sample eval’s number as final.	Every prompt fix so far had been validated only against sample_messages.csv, and repeatedly reading those same 30 examples to derive rules creates real overfitting risk even while trying to stay general. It surfaced a real gap (under-weighted “not urgent” framing) the 30-sample set hadn’t exposed. Rejected: trusting the 30-sample number alone.
Synthetic test images/voice were generated locally and free (Pillow, Windows SAPI TTS), not via an image-gen/TTS API.	This is throwaway evaluation input, not a submission deliverable — only exercising the same OCR/ASR/vision code paths matters, not media realism. Rejected: a generation API (added cost/key/latency for eval data discarded after scoring).
The key_phrase grounding field was implemented, tested, and reverted after a measured regression (93.33%→86.67% action on the 30-sample eval).	Not every plausible-sounding grounding technique helps — the categorical internal-assessment fields worked, but adding free-text extraction on top of them didn’t. A cheap 30-sample check caught this before it reached the submission file. Rejected: keeping the change on the strength of the idea alone, without re-measuring.

7. Iterative Development — The Eval Loop

The “build → run → inspect failures → fix → rerun” loop, in order. This is the process an AI Judge interview is likely to probe most directly.

#	Iteration
1	First `route --limit 3` test finished suspiciously fast with 0/3 succeeding. Root cause: a single SQLite connection shared across ThreadPoolExecutor workers (“objects created in a thread...”). Fixed by giving each worker its own connection.
2	Re-ran → 3/3 succeeded. Manually read the output against source text before scaling up — confirmed grounded reasoning (e.g. caught an OTP-phishing message disguised as personal chat by matching a prior identical message the same user had reported). Ran the full 110-message batch.
3	Discovered sample_messages.csv uses a disjoint message_id namespace (sample_msg_*) from messages.csv (msg_*) — comparing output.csv to it by id always scores 0/30. Built a script that runs the router on sample_messages.csv’s own input fields instead. Baseline: 90% action, 63% message_type, 49% evidence F1.
4	Found one real gap (opted-in repeat promotions over-muted for repetition alone) and fixed it with a general rule, not a special case. Action held at 90% (miss moved, not eliminated — read as n=30 noise) — deliberately stopped tuning to avoid overfitting the visible set.
5	User asked specifically to improve message_type accuracy. Read the actual text of every miss in the worst categories (event, promotion, greeting) and derived 4 generalizable disambiguation rules from real conventions in the data. Result: message_type 63%→90%.
6	Built a genuinely held-out 19-case synthetic set (never used to tune the prompt). First run: 74% action / 79% type — notably lower, and revealing: personal messages over-triaged to notify even when text explicitly said “no rush”. Fixed with one more general rule; re-ran: 84%/84%, no regression on the 30-sample set.
7	User pattern-spotted the digest→notify bias and the confidence/correctness correlation from reviewing results directly. Added internal reasoning fields + a default-to-digest-under-uncertainty rule. Validated cheaply first (7 previously-wrong cases: 3 fixed, 0 type regressions) before the full 30-sample check: 93.33%/93.33%, no regression.
8	Tried extending type_candidates with a key_phrase grounding field. Result: a real regression (93.33%→86.67% action). Reverted immediately, restoring the best-validated version, and reclassified the full 110-message set with it.

8. Testing & Validation Strategy

Three independent signals were used together, deliberately, rather than trusting any single one:

- **Official 30 solved examples** (dataset/sample_messages.csv) — the primary, always-available local proxy. Scored via a purpose-built harness (code/evaluation/) that validates output schema, flags degenerate label distributions, and prints an action/message_type confusion matrix.
- **Genuinely held-out synthetic set** (19 cases: 13 text + 3 Pillow-drawn images + 3 Windows-SAPI-synthesized voice notes) built specifically to check generalization, since the prompt had never seen these cases. This is what caught the urgency-framing gap the 30-sample set missed.
- **Targeted cheap re-checks** — before spending on a full re-run after a prompt change, reclassify only the previously-known-wrong cases first (7 messages, not 30 or 110) to get a fast, cheap read on whether an idea is working before committing further spend.
- **Schema and sanity checks** — exact column order, allowed action/message_type value sets, confidence range, evidence ids validated against the actual user's message_history (not invented, not another user's history), and a non-degenerate action distribution.

An HTML report with the synthetic set's 19 cases (embedded images/audio, expected-vs-actual, pass/fail) plus an illustrative rule-based “try it yourself” simulator was also built and published — clearly labeled as a simplified client-side approximation, not a live model call, since the artifact platform available in this session offered no capability to call the Anthropic API from a published page without risking an API-key leak.

9. Cost & Resource Management

- Model choice (Claude Sonnet 5) was made from an explicit cost/quality/speed estimate before any spend, given the small data volume (20 images, 13 voice notes, 110 messages) made even Opus-tier cost negligible in absolute terms — Sonnet was chosen for quality headroom at lower cost, not because it was the only affordable option.
- Local, free where it mattered: Whisper ASR (no per-call API cost) and synthetic test-media generation (Pillow + Windows SAPI TTS) — both throwaway/development-time costs kept at zero.
- Content-hash caching across the whole pipeline meant re-running preprocessing after a code fix never re-paid for unchanged media.
- A real cost-tracking bug was found and fixed mid-session: the cache-status command's “cumulative spend” figure summed only the current cache files, which repeated - -force reclassification runs overwrite — so it silently under-reported total spend across multiple full re-runs. Caught by manually reconciling every individual run's reported cost, then communicated transparently before continuing to spend.

10. Final Results

action accuracy	93.33% (28/30, official sample set)
message_type accuracy	93.33% (28/30, official sample set)
evidence F1 (avg)	~48–52%, consistent across evaluation runs
synthetic held-out set (final)	84% action / 84% message_type (19 cases, never used to tune)
output.csv	110/110 rows written, 0 fallback rows, schema-exact
action distribution	mute 55 / notify 33 / digest 22 (non-degenerate)

11. Known Limitations & Honest Open Items

- **evidence_message_ids F1 (~48–52%)** lags action/message_type accuracy — the current heuristic retrieval works but doesn't always match the exact ids a grader would consider most relevant. Not tuned further given overfitting risk on a 30-example gold set and limited remaining budget.
- **spam vs scam, event vs urgent** stayed genuinely ambiguous across both eval sets even after the message_type fixes — read as inherent category-boundary fuzziness (both readings defensible on the actual content) rather than a fixable bug.
- **Whisper model size** is base (fastest, CPU-only) — a larger model might improve transcript quality but wasn't attempted; spot-checked transcripts were already qualitatively good, and budget was tight by the end of the session.
- **The internal reasoning fields are a heuristic improvement, not a proof** — they measurably helped on two independent eval sets, which is meaningful evidence at this scale, but n=30/n=19 eval sets can't rule out further edge cases the hidden grading set might contain.

12. Submission Artifacts

- **code.zip** — the code/ directory only (main.py, config.py, db.py, cache.py, image_pipeline.py, audio_pipeline.py, router.py, query.py, evaluation/), excluding .venv/, data/, __pycache__/. Includes a self-contained code/README.md since code.zip's contents are all a reviewer sees.
- **output.csv** — final predictions for all 110 messages in dataset/messages.csv.
- **log.txt** — the AGENTS.md-mandated chat transcript, appended after every turn for the entire session, including the onboarding agreement and every decision described in this report.

This report itself, plus the full CLAUDE.en.md/CLAUDE.ko.md project briefs (architecture principles, complete eval loop log, complete decision log, open questions) and the synthetic-test HTML report are available in the repository for deeper review, though only code/ ships in code.zip per the submission instructions.